Ngày 1 (làm lại): Value và reference

Oct 17, 2026 · @Hoàng Tử Hói

Tổ chức repo và quy trình mỗi ngày

Phần này dùng cho cả 6 tháng. Làm đúng một lần từ hôm nay thì về sau chỉ việc lặp lại. Bộ bài tập ngày 1 bên dưới là bộ **mới**, khác bộ cũ, vì em đã xem đáp án bộ cũ rồi.

Cài đặt một lần

Trong thư mục repo mới, chạy:

```
git config user.name "Tên của em"
git config user.email "email-ca-nhan@gmail.com"   # email cá nhân, không dùng
email công ty
node -v                                           # cần 20 trở lên
```

Tắt GitHub Copilot hoặc AI autocomplete trong VS Code cho repo này.

Cấu trúc thư mục

Tên thư mục và file dùng chữ thường, không dấu, không khoảng trắng (`day-01` , không phải `Ngày 1`). Tên có dấu và khoảng trắng làm Git và terminal hiển thị sai, và đây là quy ước chung trong mọi công ty.

```
frontend-2026/
├── README.md              # mục tiêu 6 tháng + bảng tiến độ từng ngày
├── .gitignore             # có 1 dòng: node_modules
└── js-core/
    └── day-01/
        ├── predictions.md # bảng dự đoán (mẫu bên dưới)
        ├── snippets.js    # dán các đoạn code phần A vào đây để chạy
        ├── exercises.js   # bài code tay phần B
        ├── react-bugs.md  # phần C
        └── log.md         # nhật ký
```

Ngày 2 sẽ là `js-core/day-02/` , và cứ thế tiếp tục.

Mẫu `predictions.md`

Mỗi câu một dòng, đủ 4 cột. Cột Dự đoán viết TRƯỚC khi chạy code và không sửa lại sau đó, kể cả khi sai.

```
| Câu | Dự đoán | Kết quả thật | Vì sao |
| --- | --- | --- | --- |
| VD1 | 20 | 10 | Sai. Loại 1 (gán biến): b = 20 chỉ đổi hộp b, a vẫn giữ 10.
|
| VD2 | "Lan" | "Lan" | Đúng. user2 = user1 copy địa chỉ; user2.name = ... là
loại 2, sửa nhà chung. |
```

Mẫu `log.md`

```
# Ngày 1 - 17/10/2026
- Hôm nay hiểu: ...
- Còn mơ hồ: ...
- Ngày mai: ...
```

Quy trình mỗi ngày

1. `git pull`, tạo thư mục `js-core/day-XX/`.
2. Đọc bài giảng, gõ lại ví dụ vào một file nháp rồi chạy bằng `node`.
3. Phần A: điền cột Dự đoán cho tất cả các câu, sau đó mới chạy `node js-core/day-XX/snippets.js`, rồi điền 2 cột còn lại.
4. Phần B: mỗi hàm kèm `console.log` kiểm tra, ghi kết quả mong đợi ở comment cuối dòng.
5. Phần C và `log.md`.
6. Commit và push, rồi gửi **link thư mục ngày đó** vào thread, ví dụ `github.com/<user>/<repo>/tree/main/js-core/day-01`.

Repo phải để public để thầy đọc được.

Bài giảng (40 phút)

Đọc lại phần Bài giảng trong tài liệu Ngày 1 cũ, mục 1 đến 7. Chỉ đọc phần Bài giảng, đừng mở phần Nhận xét ở cuối. Gõ lại từng ví dụ vào file nháp và chạy. Sau đó học thuộc bản tóm tắt dưới đây.

Hộp và căn nhà

Biến là cái hộp. Primitive (`string`, `number`, `boolean`, `null`, `undefined`, `bigint`, `symbol`) nằm ngay trong hộp. Object, array, function là căn nhà ở chỗ khác; trong hộp chỉ có tờ giấy ghi địa chỉ.

Hai loại thao tác

Loại	Trông như thế nào	Ảnh hưởng
1. Gán biến	`x = ...`	Chỉ hộp `x` đổi. Không ai khác bị ảnh hưởng.
2. Sửa nhà	`x.a = ...`, `x[0] = ...`, `x.push()`, `x.sort()`, `x.qty++`	Căn nhà bị sửa. Mọi biến trỏ vào nhà đó đều thấy.

Cái gì tạo nhà mới, cái gì trả nhà cũ

Tạo nhà mới	Trả lại nhà cũ
`{}`, `[]`, `{ ...x }`, `[...x]`, `map`, `filter`, `slice`, `toSorted`, `structuredClone`	`const b = a`, `find`, `arr[0]`, `obj.address`, truyền vào hàm `f(a)`

Lưu ý: spread, `map`, `filter`, `slice` chỉ làm mới **lớp ngoài**. Object bên trong vẫn là nhà cũ. Chỉ `structuredClone` copy sâu.

Method sửa nhà vs method tạo nhà mới

Sửa mảng gốc (tránh dùng với state React)	Trả mảng mới (an toàn)
`push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`, `fill`	`concat`, `map`, `filter`, `slice`, `toSorted`, `toReversed`, `toSpliced`, `with`

So sánh

`===`, `==`, `Object.is`, `includes`, `Set` đều so object bằng **địa chỉ**. Hai object viết giống hệt nhau vẫn là hai nhà khác nhau. React (`setState`, dependency của `useEffect`) cũng so bằng địa chỉ.

Cách đọc code

Với mỗi dòng, tự hỏi: "Loại 1 hay loại 2?". Nếu loại 2, hỏi tiếp: "Căn nhà này còn ai trỏ vào?". Khi chưa chắc, dán code vào pythontutor.com để xem mũi tên.

Bài tập

Phần A: Đoán output (25 phút), file `snippets.js` và `predictions.md`

```
// A1
let name = "an";
name.toUpperCase();
console.log(name);

// A2
const nums = [1, 2];
const copy = nums;
copy[0] = 99;
console.log(nums[0]);

// A3
console.log({} === {}, "5" == 5, "5" === 5, typeof undefined);

// A4
const p = { age: 20 };
function birthday(person) {
  person.age++;
  person = { age: 0 };
}
birthday(p);
console.log(p.age);

// A5
const users = [{ n: "A" }];
const clone = users.slice();
clone[0].n = "B";
clone.push({ n: "C" });
console.log(users[0].n, users.length);

// A6
console.log(typeof [], typeof null, typeof (() => {}), Array.isArray({}));

// A7
const t = { x: 1 };
console.log(new Set([t, t, { x: 1 }]).size);

// A8
const cfg = { theme: { dark: false } };
```

```
const c2 = { ...cfg };
const c3 = structuredClone(cfg);
c2.theme.dark = true;
console.log(cfg.theme.dark, c3.theme.dark);
```

Phần B: Code tay (40 phút), file `exercises.js`

Không hàm nào được sửa dữ liệu đầu vào. Mỗi hàm có ít nhất 3 dòng `console.log`, trong đó có dòng chứng minh input cũ không đổi. Cuối mỗi dòng log, ghi kết quả mong đợi bằng comment, ví dụ `console.log(x) // 3`.

1. `pick(obj, keys)`: trả về object mới chỉ gồm các key trong mảng `keys`. Ví dụ `pick({ a: 1, b: 2, c: 3 }, ["a", "c"])` ra `{ a: 1, c: 3 }`.
2. `renameTag(user, oldTag, newTag)`: `user` có dạng `{ name, tags: ["js", "css"] }`. Trả về user mới với tag đã đổi tên. Chứng minh `user.tags` cũ không đổi.
3. `addToCart(cart, product)`: `cart` là mảng `{ id, name, qty }`. Nếu sản phẩm đã có thì tăng `qty` thêm 1, nếu chưa có thì thêm `{ ...product, qty: 1 }` vào cuối. Các item không liên quan phải giữ nguyên địa chỉ cũ (chứng minh bằng `===`).
4. `shallowEqual(a, b)`: viết lại **từ trí nhớ**, không mở bài cũ. Có 5 test: ít nhất 2 ca ra `true`, và ca `{ a: undefined }` so với `{ b: 1 }` phải ra `false`.

Phần C: Bug React (20 phút), file `react-bugs.md`

Với mỗi bug: giải thích nguyên nhân trong 2-3 câu, bắt buộc dùng từ "địa chỉ", rồi viết code đã sửa.

```
// C1: Bấm "Chuyển đến HCM". Tên thành phố ở phần header cập nhật,
// nhưng component AddressCard (đã bọc React.memo) vẫn hiển thị "HN". Vì sao?
function Profile() {
  const [user, setUser] = useState({ name: "An", address: { city: "HN" } });

  function moveTo(city) {
    const next = { ...user };
    next.address.city = city;
    setUser(next);
  }

  return (
    <>
      <h1>{user.name} - {user.address.city}</h1>
      <AddressCard address={user.address} />
      <button onClick={() => moveTo("HCM")}>Chuyển đến HCM</button>
    </>
  );
```

```
}
const AddressCard = React.memo(({ address }) => <p>{address.city}</p>);

// C2: Bấm "Sắp xếp theo giá" nhưng danh sách trên màn hình không đổi thứ tự.
Vì sao?
function ProductList({ data }) {
  const [list, setList] = useState(data);

  function sortByPrice() {
    const sorted = list.sort((a, b) => a.price - b.price);
    setList(sorted);
  }
  // ...render list và nút "Sắp xếp theo giá"
}
```

Nộp bài và tiêu chí đạt

- [ ] Repo public, đã có `README.md` và `.gitignore`
- [ ] `js-core/day-01/` có đủ 5 file: `predictions.md`, `snippets.js`, `exercises.js`, `react-bugs.md`, `log.md`
- [ ] `predictions.md` đủ 4 cột cho cả 8 câu, cột Dự đoán giữ nguyên dù sai
- [ ] Commit bằng email cá nhân, message: `docs(js-core): day 01 value vs reference`
- [ ] Gửi link thư mục `day-01` vào thread

Tiêu chí đạt để sang ngày 2:

1. Phần A đúng ít nhất 6/8 ngay ở cột Dự đoán, và cột Vì sao câu nào cũng nói được "loại 1 hay loại 2" hoặc "nhà mới hay nhà cũ".
2. Phần B: 4 hàm chạy đúng, không mutate input, có log chứng minh.
3. Phần C: giải thích đúng nguyên nhân cả hai bug.

Chưa đạt thì không sao. Thầy sẽ chỉ ra đúng chỗ hiểu sai và giao bài ôn ngắn trước khi đi tiếp.